

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Question 1:

Normalization of Order Relation to 3NF

Given relation:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

1. Identify the Functional Dependencies (FDs)

Assume:

- Each OrderID identifies one customer/order.
- Each CustID identifies one customer.
- Each ProdID identifies one product.
- The quantity ordered depends on the combination of OrderID and ProdID.

Therefore, the functional dependencies are:

1. $\text{OrderID} \rightarrow \text{CustID}, \text{CustName}$
2. $\text{CustID} \rightarrow \text{CustName}$
3. $\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$
4. $\text{OrderID}, \text{ProdID} \rightarrow \text{Qty}$

The candidate key is:

(OrderID, ProdID)

because together they uniquely identify each order item.

2. Check for 1NF

The relation is already in **1NF** because all attributes contain atomic/single values.

So:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

3. Convert to 2NF

There are partial dependencies because some non-key attributes depend only on part of the composite key.

- $\text{OrderID} \rightarrow \text{CustID}, \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$

Therefore, decompose the relation into:

ORDER(OrderID, CustID, CustName)
PRODUCT(ProdID, ProdName, Price)
ORDER_ITEM(OrderID, ProdID, Qty)

Now, all non-key attributes depend on the whole key of their respective relations.

4. Convert to 3NF

In the ORDER relation:

$\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$

This is a **transitive dependency**, because CustName depends on CustID rather than directly on OrderID.

So decompose ORDER into:

ORDER(OrderID, CustID)
CUSTOMER(CustID, CustName)

The final 3NF relations are:

Relation	Attributes	Primary Key
CUSTOMER	CustID, CustName	CustID
ORDER	OrderID, CustID	OrderID
PRODUCT	ProdID, ProdName, Price	ProdID
ORDER_ITEM	OrderID, ProdID, Qty	(OrderID, ProdID)

5. Final 3NF Schema

CUSTOMER(CustID, CustName)
ORDER(OrderID, CustID)
PRODUCT(ProdID, ProdName, Price)
ORDER_ITEM(OrderID, ProdID, Qty)

Foreign keys:

- ORDER.CustID $\rightarrow$ CUSTOMER.CustID
- ORDER_ITEM.OrderID $\rightarrow$ ORDER.OrderID
- ORDER_ITEM.ProdID $\rightarrow$ PRODUCT.ProdID

Conclusion:

The original relation contains partial and transitive dependencies, causing update, insertion, and deletion anomalies. By decomposing it into CUSTOMER, ORDER, PRODUCT, and ORDER_ITEM, the database is normalized to 3NF, reducing redundancy and maintaining data consistency.

Question 2:

Hospital Relation – Normalization to BCNF

Given relation:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

1. Identify Functional Dependencies

Assume:

- Each PID uniquely identifies a patient.
- Each DocID uniquely identifies a doctor.
- Each WardNo uniquely identifies a ward.
- A patient is assigned to one doctor and one ward.

Therefore:

1. $PID \rightarrow PName, DocID, WardNo$
2. $DocID \rightarrow DocName, Specialization$
3. $WardNo \rightarrow WardName$

The candidate key is:

PID

because PID determines all the other attributes.

2. Identify Anomalies

The original relation has the following problems:

Update anomaly:

If a doctor's specialization changes, we must update it in every patient record associated with that doctor. Missing one update causes inconsistent data.

Insertion anomaly:

We cannot store a new doctor's details unless the doctor is assigned to at least one patient.

Deletion anomaly:

If the only patient assigned to a particular doctor is deleted, the doctor's information may also be lost.

Similarly, deleting the only patient in a ward could remove the ward's information.

3. Check BCNF

A relation is in **BCNF** if, for every non-trivial FD:

$X \rightarrow Y$, X must be a superkey.

In the original relation:

$PID \rightarrow PName, DocID, WardNo$

PID is a candidate key, so this satisfies BCNF.

But:

$DocID \rightarrow DocName, Specialization$

$DocID$ is **not** a superkey of the original relation.

Therefore, the relation **violates BCNF**.

Also:

$WardNo \rightarrow WardName$

$WardNo$ is not a superkey, so this also violates BCNF.

4. Decompose into BCNF

Separate the doctor information:

DOCTOR($DocID, DocName, Specialization$)

Here:

$DocID \rightarrow DocName, Specialization$

$DocID$ is the primary key, so this relation is in BCNF.

Separate the ward information:

WARD($WardNo, WardName$)

Here:

$WardNo \rightarrow WardName$

WardNo is the primary key, so this relation is in BCNF.

The remaining patient information becomes:

PATIENT(PID, PName, DocID, WardNo)

Here:

$PID \rightarrow PName, DocID, WardNo$

PID is the primary key, so this relation is also in BCNF.

5. Final BCNF Schema

Relation	Attributes	Primary Key
PATIENT	PID, PName, DocID, WardNo	PID
DOCTOR	DocID, DocName, Specialization	DocID
WARD	WardNo, WardName	WardNo

Foreign Keys:

- PATIENT.DocID $\rightarrow$ DOCTOR.DocID
- PATIENT.WardNo $\rightarrow$ WARD.WardNo

Final Answer:

The original Patient relation violates BCNF due to the dependencies $DocID \rightarrow DocName, Specialization$ and $WardNo \rightarrow WardName$, since DocID and WardNo are not superkeys. Decomposing it into PATIENT, DOCTOR, and WARD removes redundancy and anomalies, and all three resulting relations satisfy BCNF.

Question 3:

Case Study: University Enrollment – 2NF Decomposition

Given relation:

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

1. Identify Functional Dependencies

Assume:

- Each SID uniquely identifies a student.
- Each CourseID uniquely identifies a course.
- A student's grade depends on the particular student and course combination.

Therefore, the functional dependencies are:

1. $SID \rightarrow SName$
2. $CourseID \rightarrow CourseName, Faculty$
3. $(SID, CourseID) \rightarrow Grade$

2. Find the Candidate Key

A student can enroll in multiple courses, and a course can have multiple students. Therefore, $(SID, CourseID)$ uniquely identifies an enrollment.
Candidate Key = $(SID, CourseID)$

3. Check for 1NF

The relation is already in 1NF because all attributes contain atomic values. However, it is not in 2NF.

4. Identify Partial Dependencies

Since the candidate key is composite $(SID, CourseID)$:

- $SID \rightarrow SName$
SName depends only on part of the key.
- $CourseID \rightarrow CourseName, Faculty$
CourseName and Faculty depend only on part of the key.

These are partial dependencies, so the relation violates 2NF.

Only:

$$(SID, CourseID) \rightarrow Grade$$

depends on the complete composite key.

5. Decompose into 2NF

Separate student details:

STUDENT($SID, SName$)

- Primary Key: **SID**

Separate course details:

COURSE($CourseID, CourseName, Faculty$)

- Primary Key: **CourseID**

Keep the attributes that depend on the complete key:

ENROLLMENT($SID, CourseID, Grade$)

- Primary Key: **(SID, CourseID)**

6. Final 2NF Relations

Relation	Attributes	Primary Key
STUDENT	SID, SName	SID
COURSE	CourseID, CourseName, Faculty	CourseID
ENROLLMENT	SID, CourseID, Grade	(SID, CourseID)

Foreign keys:

- ENROLLMENT.SID $\rightarrow$ STUDENT.SID
- ENROLLMENT.CourseID $\rightarrow$ COURSE.CourseID

Conclusion:

The original ENROLLMENT relation has a composite key (SID, CourseID) and contains partial dependencies. By decomposing it into STUDENT, COURSE, and ENROLLMENT, all partial dependencies are removed and the resulting relations are in 2NF.

Question 4:

Airline Booking – Normalization up to BCNF

Given relation:

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

1. Identify Functional Dependencies

Assume:

- Each PassID uniquely identifies a passenger.
- A FlightNo + Date identifies a particular flight instance.
- For a particular flight instance, each Seat is assigned to at most one passenger.
- Flight details such as departure and arrival cities depend on the flight.

Functional dependencies:

1. PassID $\rightarrow$ PassName
2. FlightNo $\rightarrow$ DeptCity, ArrCity
3. (FlightNo, Date, Seat) $\rightarrow$ PassID
4. (FlightNo, Date, PassID) $\rightarrow$ Seat

The candidate keys can therefore be:

- (FlightNo, Date, Seat)

- (FlightNo, Date, PassID)

2. Check 1NF

All attributes contain atomic values, so the relation is in 1NF.

3. Convert to 2NF

There are partial dependencies:

$$\text{PassID} \rightarrow \text{PassName}$$

and

$$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$$

because these attributes depend on only part of the composite keys.

Decompose into:

PASSENGER(PassID, PassName)
FLIGHT(FlightNo, DeptCity, ArrCity)

The remaining relation is:

BOOKING(FlightNo, Date, PassID, Seat)

4. Check for 3NF

In BOOKING:

- (FlightNo, Date, PassID) $\rightarrow$ Seat
- (FlightNo, Date, Seat) $\rightarrow$ PassID

Both determinants are candidate keys.

Therefore, there is no transitive dependency violating 3NF.

5. Check BCNF

A relation is in BCNF if every determinant is a superkey.

PASSENGER

$$\text{PassID} \rightarrow \text{PassName}$$

PassID is the primary key.

BCNF

FLIGHT

$$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$$

FlightNo is the primary key.

BCNF

BOOKING

- $(\text{FlightNo}, \text{Date}, \text{PassID}) \rightarrow \text{Seat}$
- $(\text{FlightNo}, \text{Date}, \text{Seat}) \rightarrow \text{PassID}$

Both determinants are candidate keys and therefore superkeys.

BCNF

BCNF Decomposition:

Relation	Attributes	Primary Key
PASSENGER	PassID, PassName	PassID
FLIGHT	FlightNo, DeptCity, ArrCity	FlightNo
BOOKING	FlightNo, Date, PassID, Seat	(FlightNo, Date, PassID)

Foreign Keys:

- $\text{BOOKING.PassID} \rightarrow \text{PASSENGER.PassID}$
- $\text{BOOKING.FlightNo} \rightarrow \text{FLIGHT.FlightNo}$

Conclusion

The original BOOKING relation contains dependencies that cause redundancy. Decomposing it into PASSENGER, FLIGHT, and BOOKING removes partial dependencies and ensures that every determinant in the resulting relations is a superkey. Hence, the final decomposition is in BCNF.

Question 5:

HR System – Normalization to 3NF

Consider the HR relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, ManagerPhone, JobID, JobTitle, Salary)

Assume:

- Each employee has a unique EmpID.
- Each department has a unique DeptID.
- Each department has one manager.
- Each job has a unique JobID.

1. Functional Dependencies

The main functional dependencies are:

1. $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{JobID}, \text{Salary}$
2. $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$
3. $\text{ManagerID} \rightarrow \text{ManagerName}, \text{ManagerPhone}$
4. $\text{JobID} \rightarrow \text{JobTitle}$

Therefore, we can write:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{JobID}, \text{Salary}$
 $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$
 $\text{ManagerID} \rightarrow \text{ManagerName}, \text{ManagerPhone}$
 $\text{JobID} \rightarrow \text{JobTitle}$

2. Candidate Key

Since EmpID uniquely identifies an employee:

Candidate Key = EmpID

3. Identify Transitive Dependencies

A transitive dependency occurs when:

$A \rightarrow B$ and $B \rightarrow C$, therefore $A \rightarrow C$.

Dependency 1

$\text{EmpID} \rightarrow \text{DeptID}$

and

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Therefore:

$\text{EmpID} \rightarrow \text{DeptName}, \text{ManagerID}$

This is a transitive dependency.

Dependency 2

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID}$

and

$\text{ManagerID} \rightarrow \text{ManagerName}, \text{ManagerPhone}$

Therefore:

$\text{EmpID} \rightarrow \text{ManagerName}, \text{ManagerPhone}$

This is also a transitive dependency.

Dependency 3

$\text{EmpID} \rightarrow \text{JobID}$

and

$\text{JobID} \rightarrow \text{JobTitle}$

Therefore:

$\text{EmpID} \rightarrow \text{JobTitle}$

This is another transitive dependency.

4. Normalize to 3NF

Step 1: Employee Relation

Keep employee-specific information:

EMPLOYEE(EmpID, EmpName, DeptID, JobID, Salary)

- Primary Key: EmpID

Step 2: Department Relation

Separate department information:

DEPARTMENT(DeptID, DeptName, ManagerID)

- Primary Key: DeptID

Step 3: Manager Relation

Separate manager information:

MANAGER(ManagerID, ManagerName, ManagerPhone)

- Primary Key: ManagerID

Step 4: Job Relation

Separate job information:

JOB(JobID, JobTitle)

- Primary Key: JobID

5. Final 3NF Schema

Relation	Attributes	Primary Key
EMPLOYEE	EmpID, EmpName, DeptID, JobID, Salary	EmpID
DEPARTMENT	DeptID, DeptName, ManagerID	DeptID
MANAGER	ManagerID, ManagerName, ManagerPhone	ManagerID
JOB	JobID, JobTitle	JobID

Foreign keys:

- $\text{EMPLOYEE.DeptID} \rightarrow \text{DEPARTMENT.DeptID}$
- $\text{EMPLOYEE.JobID} \rightarrow \text{JOB.JobID}$
- $\text{DEPARTMENT.ManagerID} \rightarrow \text{MANAGER.ManagerID}$

Conclusion

The original HR schema contains several transitive dependencies, such as:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

$\text{EmpID} \rightarrow \text{JobID} \rightarrow \text{JobTitle}$

These cause update, insertion, and deletion anomalies. By decomposing the schema into EMPLOYEE, DEPARTMENT, MANAGER, and JOB, the transitive dependencies are removed and the resulting relations satisfy 3NF.

Question 6:

Library Case – MVD Identification and 4NF Decomposition

Given relation:

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

1. Identify Functional Dependencies

Assume:

- $\text{MemberID} \rightarrow \text{MemberName}$
- $\text{BookID} \rightarrow \text{Title, Author, Genre}$
- $(\text{MemberID, BookID}) \rightarrow \text{BorrowDate}$

The candidate key is:

(MemberID, BookID)

2. Identify Multivalued Dependencies (MVDs)

Suppose a member can borrow multiple books, and each book has its own details.

The important MVD is:

$$\text{MemberID} \twoheadrightarrow \text{BookID}$$

This means a member can be associated with multiple BookIDs independently.

However, to demonstrate a genuine 4NF violation, suppose the library also records another independent multivalued attribute, such as **FavoriteGenre**, giving:

$$\text{MemberID} \twoheadrightarrow \text{BookID}$$

and

$$\text{MemberID} \twoheadrightarrow \text{FavoriteGenre}$$

in the same relation.

But the given relation does not contain a separate independent multivalued attribute. Therefore, strictly speaking, there is no non-trivial MVD violation requiring a 4NF decomposition in the exact schema given.

3. If the Intended Case Has Independent MVDs

If the intended relation is:

LIBRARY(MemberID, MemberName, BookID, Genre)

and a member can have multiple books and multiple genres independently:

$$\text{MemberID} \twoheadrightarrow \text{BookID}$$

$$\text{MemberID} \twoheadrightarrow \text{Genre}$$

Then the relation violates **4NF** because MemberID is not a superkey.

Decompose into:

MEMBER_BOOK(MemberID, BookID)

MEMBER_GENRE(MemberID, Genre)

and:

MEMBER(MemberID, MemberName)

Relation	Attributes	Key
MEMBER	MemberID, MemberName	MemberID
MEMBER_BOOK	MemberID, BookID	(MemberID, BookID)
MEMBER_GENRE	MemberID, Genre	(MemberID, Genre)

Question 7:

Telecom Billing Schema

Consider:

BILLING(BillNo, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallID, CallDate, Duration, CallCharge)

1. Functional Dependencies

Assume the following business rules:

1. BillNo $\rightarrow$ CustomerID, PlanID, MonthlyCharge
2. CustomerID $\rightarrow$ CustomerName, PhoneNo
3. PlanID $\rightarrow$ PlanName, MonthlyCharge
4. CallID $\rightarrow$ CallDate, Duration, CallCharge
5. (BillNo, CallID) $\rightarrow$ all attributes of the billing record

Therefore, the main FDs are:

1. BillNo $\rightarrow$ CustomerID, PlanID
2. CustomerID $\rightarrow$ CustomerName, PhoneNo
3. PlanID $\rightarrow$ PlanName, MonthlyCharge
4. CallID $\rightarrow$ CallDate, Duration, CallCharge
5. (BillNo, CallID) $\rightarrow$ CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallDate, Duration, CallCharge

2. Candidate Key

A single BillNo cannot uniquely identify every row because one bill can contain multiple call records.

Therefore, we need:

(BillNo, CallID)

to uniquely identify each tuple.

Candidate Key:

{BillNo, CallID}

Since this is the only minimal attribute combination that identifies every tuple:

Candidate Keys = {BillNo, CallID}

3. Primary Key

From the candidate keys, choose:

Primary Key = (BillNo, CallID)

This combination uniquely identifies each billing record.

4. Prime and Non-Prime Attributes

Prime attributes

Attributes that form part of a candidate key:

- BillNo
- CallID

Non-prime attributes

- CustomerID
- CustomerName
- PhoneNo
- PlanID
- PlanName
- MonthlyCharge
- CallDate
- Duration
- CallCharge

5. Dependency Analysis

Functional Dependency	Type
BillNo $\rightarrow$ CustomerID, PlanID	Partial dependency
CustomerID $\rightarrow$ CustomerName, PhoneNo	Transitive dependency
PlanID $\rightarrow$ PlanName, MonthlyCharge	Transitive dependency through BillNo
CallID $\rightarrow$ CallDate, Duration, CallCharge	Partial dependency
(BillNo, CallID) $\rightarrow$ all billing details	Full functional dependency

Why partial dependency?

The candidate key is:

(BillNo, CallID)

But:

BillNo $\rightarrow$ CustomerID, PlanID

and

$\text{CallID} \rightarrow \text{CallDate}, \text{Duration}, \text{CallCharge}$

So some non-key attributes depend on only part of the composite key.

Why transitive dependency?

We have:

$\text{BillNo} \rightarrow \text{CustomerID}$

and:

$\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}$

Therefore:

$\text{BillNo} \rightarrow \text{CustomerName}, \text{PhoneNo}$

Similarly:

$\text{BillNo} \rightarrow \text{PlanID}$

and:

$\text{PlanID} \rightarrow \text{PlanName}, \text{MonthlyCharge}$

Therefore:

$\text{BillNo} \rightarrow \text{PlanName}, \text{MonthlyCharge}$

These are transitive dependencies.

6. Final Answer

Candidate Key:

(BillNo, CallID)

Primary Key:

(BillNo, CallID)

Functional Dependencies:

- $\text{BillNo} \rightarrow \text{CustomerID}, \text{PlanID}$
- $\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}$
- $\text{PlanID} \rightarrow \text{PlanName}, \text{MonthlyCharge}$
- $\text{CallID} \rightarrow \text{CallDate}, \text{Duration}, \text{CallCharge}$
- $(\text{BillNo}, \text{CallID}) \rightarrow \text{all attributes}$

Partial Dependencies:

- $\text{BillNo} \rightarrow \text{CustomerID}, \text{PlanID}$
- $\text{CallID} \rightarrow \text{CallDate}, \text{Duration}, \text{CallCharge}$

Transitive Dependencies:

- $\text{BillNo} \rightarrow \text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}$

- BillNo $\rightarrow$ PlanID $\rightarrow$ PlanName, MonthlyCharge

Thus, the original telecom billing schema contains **partial and transitive dependencies**, indicating that it is not in 2NF/3NF and should be decomposed during normalization.

Question 8:

Retail Inventory System – Complete Normalization

Given relation:

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Assume the following business rules:

- Each PID uniquely identifies a product.
- Each SupplierID uniquely identifies a supplier.
- Each product belongs to one category.
- A product can be stored in multiple warehouses, so Warehouse is treated as a multivalued attribute.

1. Functional Dependencies

The main FDs are:

1. PID $\rightarrow$ PName, SupplierID, Category, Price
2. SupplierID $\rightarrow$ SupplierName

There is also an MVD:

PID $\twoheadrightarrow$ Warehouse

because one product may be stored in multiple warehouses independently.

2. First Normal Form (1NF)

To satisfy 1NF, every attribute must contain atomic values.

If a product is stored in multiple warehouses, represent each warehouse separately.

The relation can therefore have the key:

(PID, Warehouse)

So:

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

with candidate key:

(PID, Warehouse)

3. Second Normal Form (2NF)

The key is (PID, Warehouse).

But:

$PID \rightarrow PName, SupplierID, Category, Price$

These attributes depend only on PID, which is only part of the composite key.

Therefore, there are partial dependencies, so the relation is not in 2NF.

Decompose into:

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price)

PRODUCT_WAREHOUSE(PID, Warehouse)

4. Third Normal Form (3NF)

In PRODUCT:

$PID \rightarrow SupplierID$

and:

$SupplierID \rightarrow SupplierName$

Therefore:

$PID \rightarrow SupplierName$

This is a transitive dependency.

So decompose further:

PRODUCT(PID, PName, SupplierID, Category, Price)

SUPPLIER(SupplierID, SupplierName)

PRODUCT_WAREHOUSE(PID, Warehouse)

5. BCNF Check

PRODUCT

$PID \rightarrow PName, SupplierID, Category, Price$

PID is the primary key.

BCNF

SUPPLIER

$SupplierID \rightarrow SupplierName$

SupplierID is the primary key.
BCNF

PRODUCT_WAREHOUSE

There is no non-trivial FD such as $PID \rightarrow Warehouse$, because a product can occur in multiple warehouses.

Its key is:

(PID, Warehouse)

BCNF

Final Normalized Schema

Relation	Attributes	Primary Key
PRODUCT	PID, PName, SupplierID, Category, Price	PID
SUPPLIER	SupplierID, SupplierName	SupplierID
PRODUCT_WAREHOUSE	PID, Warehouse	(PID, Warehouse)

Foreign Key

PRODUCT.SupplierID $\rightarrow$ SUPPLIER.SupplierID
PRODUCT_WAREHOUSE.PID $\rightarrow$ PRODUCT.PID

Final Conclusion

The original relation contains partial, transitive, and multivalued dependencies. Complete normalization decomposes it into:

PRODUCT(PID, PName, SupplierID, Category, Price)
SUPPLIER(SupplierID, SupplierName)
PRODUCT_WAREHOUSE(PID, Warehouse)

These relations eliminate redundancy and anomalies and satisfy BCNF, which is stronger than 3NF.

Question 9:

School ERP – Complete Normalization up to BCNF

Since no specific schema is given, we can take a typical poorly normalized School ERP relation and normalize it.

Given Poorly Normalized Relation

Consider:

SCHOOL_ERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks, ExamID, ExamName)

Assume:

- Each StudentID identifies one student.
- Each ClassID identifies one class.
- Each TeacherID identifies one teacher.
- Each SubjectID identifies one subject.
- Each ExamID identifies one exam.
- A student's marks are identified by the combination of student, subject, and exam.

1. Functional Dependencies

The important FDs are:

StudentID $\rightarrow$ StudentName, ClassID
ClassID $\rightarrow$ ClassName, TeacherID
TeacherID $\rightarrow$ TeacherName
SubjectID $\rightarrow$ SubjectName
ExamID $\rightarrow$ ExamName
(StudentID, SubjectID, ExamID) $\rightarrow$ Marks

Therefore, the candidate key of the original relation is:

(StudentID, SubjectID, ExamID)

because this combination uniquely identifies a student's marks for a particular subject and examination.

2. Problems in the Original Schema

Update Anomaly

If a teacher's name changes, it has to be updated in multiple student records.
For example:

TeacherID = T01 $\rightarrow$ TeacherName = "Ravi"

If the teacher teaches many students, the same information is repeated.

Insertion Anomaly

A new teacher or subject cannot easily be stored unless there is a corresponding student/exam record.

Deletion Anomaly

If the last student taking a particular subject is deleted, information about that subject may also be lost.

3. 1NF

The relation is in 1NF if all attributes contain atomic values.

So we have:

SCHOOL_ERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks, ExamID, ExamName)

4. Convert to 2NF

The candidate key is:

(StudentID, SubjectID, ExamID)

There are partial dependencies:

- StudentID $\rightarrow$ StudentName, ClassID
- SubjectID $\rightarrow$ SubjectName
- ExamID $\rightarrow$ ExamName

These depend on only part of the composite key.

So decompose into:

STUDENT

STUDENT(StudentID, StudentName, ClassID)

SUBJECT

SUBJECT(SubjectID, SubjectName)

EXAM

EXAM(ExamID, ExamName)

RESULT

RESULT(StudentID, SubjectID, ExamID, Marks)

5. Convert to 3NF

Now examine:

STUDENT(StudentID, StudentName, ClassID)

We have:

$$\text{StudentID} \rightarrow \text{ClassID}$$

and:

$$\text{ClassID} \rightarrow \text{ClassName, TeacherID}$$

So there is a transitive dependency:

$$\text{StudentID} \rightarrow \text{ClassID} \rightarrow \text{ClassName, TeacherID}$$

Also:

$$\text{TeacherID} \rightarrow \text{TeacherName}$$

Therefore, separate the class and teacher information.

STUDENT

STUDENT(StudentID, StudentName, ClassID)

CLASS

CLASS(ClassID, ClassName, TeacherID)

TEACHER

TEACHER(TeacherID, TeacherName)

The other relations remain:

SUBJECT(SubjectID, SubjectName)

EXAM(ExamID, ExamName)

RESULT(StudentID, SubjectID, ExamID, Marks)

6. Check for BCNF

A relation is in **BCNF** if for every non-trivial FD:

$$X \rightarrow Y, X \text{ must be a superkey.}$$

STUDENT

StudentID $\rightarrow$ StudentName, ClassID

StudentID is the key.

BCNF

CLASS

ClassID $\rightarrow$ ClassName, TeacherID

ClassID is the key.

BCNF

TEACHER

TeacherID $\rightarrow$ TeacherName

TeacherID is the key.

BCNF

SUBJECT

SubjectID $\rightarrow$ SubjectName

SubjectID is the key.

BCNF

EXAM

ExamID $\rightarrow$ ExamName

ExamID is the key.

BCNF

RESULT

(StudentID, SubjectID, ExamID) $\rightarrow$ Marks

The determinant is the candidate key.

BCNF

Final BCNF Design

Relation	Attributes	Primary Key
STUDENT	StudentID, StudentName, ClassID	StudentID
CLASS	ClassID, ClassName, TeacherID	ClassID
TEACHER	TeacherID, TeacherName	TeacherID
SUBJECT	SubjectID, SubjectName	SubjectID
EXAM	ExamID, ExamName	ExamID
RESULT	StudentID, SubjectID, ExamID, Marks (StudentID, SubjectID, ExamID)	

Foreign Keys

- STUDENT.ClassID $\rightarrow$ CLASS.ClassID
- CLASS.TeacherID $\rightarrow$ TEACHER.TeacherID
- RESULT.StudentID $\rightarrow$ STUDENT.StudentID
- RESULT.SubjectID $\rightarrow$ SUBJECT.SubjectID
- RESULT.ExamID $\rightarrow$ EXAM.ExamID

Justification

The original School ERP relation has partial dependencies and transitive dependencies, which cause insertion, deletion, and update anomalies.

The normalization process is:

1NF → Remove repeating/non-atomic values
↓
2NF → Remove partial dependencies
↓
3NF → Remove transitive dependencies
↓
BCNF → Ensure every determinant is a superkey

Therefore, the final design consisting of STUDENT, CLASS, TEACHER, SUBJECT, EXAM, and RESULT is a fully normalized BCNF design, with reduced redundancy and improved data consistency.

Question 10:

Movie Streaming Platform – 5NF Decomposition

For a clear 5NF example, consider the following poorly designed relation:

STREAMING(MovieID, ActorID, PlatformID)

Assume:

- A movie can have multiple actors.
- A movie can be available on multiple platforms.
- An actor may be associated with a movie on multiple platforms.
- The three-way relationship is determined by the independent pairwise relationships.

1. Identify the Join Dependency

Suppose the business rules imply that the information can be represented independently as:

- Which **actors** act in which movies
- Which **movies** are available on which platforms
- Which **actors** are associated with which platforms

Then the relation has the join dependency:

(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

This means:

$\text{STREAMING} = \text{MOVIE_ACTOR} \bowtie \text{MOVIE_PLATFORM} \bowtie \text{ACTOR_PLATFORM}$

The original three-attribute relation therefore contains a non-trivial join dependency.

2. Why Decomposition Is Required

The relation may contain redundant combinations.

For example, if a movie has:

- 2 actors
- 3 platforms

storing every possible combination can produce unnecessary repeated data.

A change to one relationship may require several tuples to be updated.

This creates redundancy and possible insertion/deletion anomalies.

3. Decompose into 5NF

Decompose the original relation into three relations:

MOVIE_ACTOR

MOVIE_ACTOR(MovieID, ActorID)

Primary Key:

(MovieID, ActorID)

MOVIE_PLATFORM

MOVIE_PLATFORM(MovieID, PlatformID)

Primary Key:

(MovieID, PlatformID)

ACTOR_PLATFORM

ACTOR_PLATFORM(ActorID, PlatformID)

Primary Key:

(ActorID, PlatformID)

4. Lossless-Join Verification

To verify the decomposition, join the three relations:

$\text{MOVIE_ACTOR} \bowtie \text{MOVIE_PLATFORM} \bowtie \text{ACTOR_PLATFORM}$

The result should reconstruct the original:

STREAMING(MovieID, ActorID, PlatformID)

Therefore:

STREAMING = MOVIE_ACTOR $\bowtie$ MOVIE_PLATFORM $\bowtie$ ACTOR_PLATFORM

provided the stated join dependency is valid.

Example

Suppose:

MOVIE_ACTOR

MovieID ActorID

M1 A1

M1 A2

MOVIE_PLATFORM

MovieID PlatformID

M1 P1

M1 P2

ACTOR_PLATFORM

ActorID PlatformID

A1 P1

A2 P2

Joining these relations reconstructs the valid combinations represented by the original relationship, assuming the business rule that the three pairwise relationships completely determine the streaming association.

5. Why It Is 5NF

A relation is in 5NF (Project-Join Normal Form) when every non-trivial join dependency is implied by the candidate keys.

After decomposition:

- MOVIE_ACTOR contains only the movie–actor relationship.
- MOVIE_PLATFORM contains only the movie–platform relationship.
- ACTOR_PLATFORM contains only the actor–platform relationship.

There is no further meaningful non-trivial join dependency that can decompose these relations without losing information.

Therefore, the decomposed relations are in 5NF.

Final Answer

Original:

STREAMING(MovieID, ActorID, PlatformID)

Join Dependency:

*(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

5NF Decomposition:

MOVIE_ACTOR(MovieID, ActorID)
MOVIE_PLATFORM(MovieID, PlatformID)
ACTOR_PLATFORM(ActorID, PlatformID)

The decomposition is lossless because joining the three decomposed relations reconstructs the original relation under the given join dependency. Thus, the final schema satisfies 5NF/PJNF.
